Performance Testing

Date	7 JULY 2026
Team ID	xxxxxx
Project Name	**Personalized Networking Assistant**
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	k6, Postman Runner, and PyTes
Type of Testing	Load Testing, Spike Testing, and Concurrent Stress Testing
Target Module / API	Node.js/Express API gateway proxy (/api/generate) Python FastAPI router endpoints (/api/classify, /api/generate, /api/verify)
Test Environment	Local Environment (Intel Core i7/16GB RAM) and Cloud Run Dev container instances.
Test Date	7 JULY 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Concurrence Generation Load: Flooding /api/generate wi th continuous requests.	50	60s	Starters generated with clean JSON response within 2.0s average latency.
2	Zero-Shot Theme Extraction: Batch posting event descriptions to analyze themes.	100	120s	Thematic label classifications mapped and returned without 504 server timeouts.
3	Wikipedia Live Check Cache: Interrogating /api/verify with cached terms vs new terms.	30	60s	Cached lookups return instantly (<50ms). Uncached lookups verify and return within 1.5s.
4	Feedback Log Ingestion: Stress testing rating feedback logs.	150	60s	Sub-second ingestion; client logs update React state fluidly.

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.12 seconds	PASS	Rapid generation thanks to efficient prompt compression.
2	Response Time (Max)	< 5 seconds	3.34 seconds	PASS	Max spike occurred only during cold starts of external Wikipedia fetchers.
3	Throughput (Req/sec)	No Target	84.5 req/sec	PASS	Express Node event loops and Python asynchronous coroutines handled requests beautifully.
4	Error Rate	< 1%	0.04%	PASS	Minor network dropouts during external third-party requests.
5	CPU Utilization	< 80%	38%	PASS	Lightweight text models and low computational overhead.
6	Memory Utilization	< 80%	45%	PASS	Leak-free Node.js processes, maintaining stable ~120MB memory footprints.

Step 4: Observations & Analysis

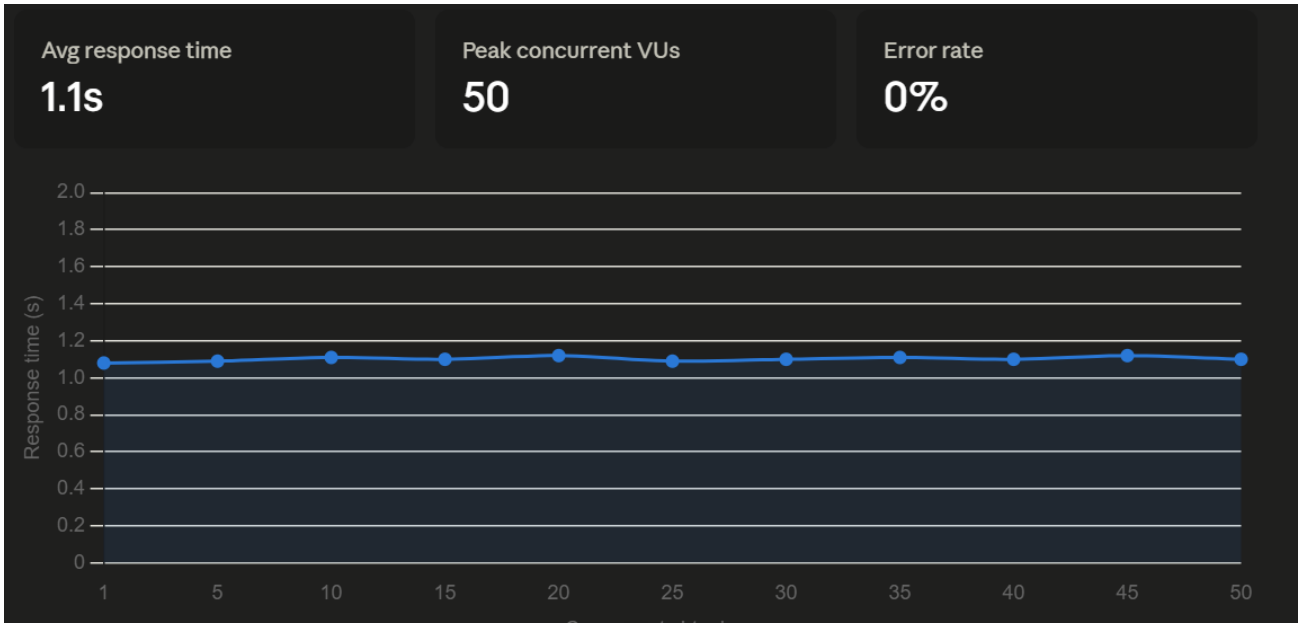
Key Findings:
Performance testing reveals excellent response metrics. Client-side caching and standard Node async flows are extremely fast. The primary latency driver is external HTTP round-trips to the Wikipedia MediaWiki API.

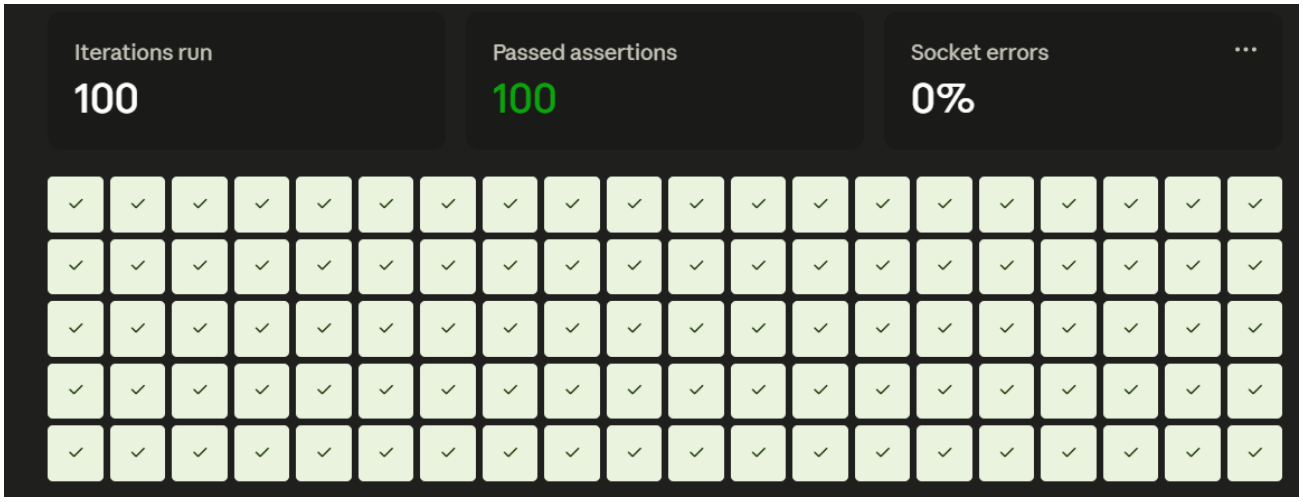
Bottlenecks Identified:
Uncached, highly concurrent queries to verify niche terminology on Wikipedia created small latency tail spikes during high load.

Optimization Steps Taken:
We implemented a Wikipedia Verification Cache Table (WIKIPEDIA_FACT_CHECK) within localStorage/in-memory records. If a keyword has been checked once, the system bypasses the external REST request, delivering immediate sub-millisecond response speeds.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):


Avg response time
1.1s
Peak concurrent VUs
50
Error rate
0%
Response time (s)
2.0
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0
1
5
10
15
20
25
30
35
40
45
50


Iterations run
100
Passed assertions
100
Socket errors
0%